Optimization for Viewshed Analysis on GPU

Yong Gao, Hao Yu*, Yu Liu, Yuehu Liu, Mingchao Liu, Yong Zhao
Institute of Remote Sensing and Geographic Information System, Peking University
Beijing, China
*Corresponding author, e-mail: yuhao006@pku.edu.cn

Abstract—**Different algorithms have been raised for viewshed analysis and measures were taken to get the compromise between performance and accuracy. The most accurate and standard algorithm is still the basic interpolation method, though its time cost is high. However, the development of Graphic Processing Unit (GPU) enables us to acquire high performance with normal PC, especially when the Compute Unified Device Architecture (CUDA) is put forward by NVIDIA for general purpose computing. In this paper, we will analyze the feasibility to map the basic interpolation method into GPU application and give our approach to achieve this goal. Further, we will introduce two critical measures in this approach: one is how to assign the data into different memory spaces on GPU according to their different access characteristics; the other is how to regularize the computing instructions and minimize branch parts in the procedure. At most, nearly 70 times speedup is reached in the experiment compared with the basic interpolation method on CPU.**

Keywords-viewshed analysis; Graphic Processing Unit (GPU); Compute Unified Device Architecture (CUDA); parallel programming

I. INTRODUCTION

Viewshed shows an area of land, water, or other environmental element that can be seen from an observer location. Viewshed analyses, which use the elevation value of each cell of the Digital Elevation Model (DEM) to determine visibility to or from a particular cell, are common functions of most GIS softwares [1]. They are widely used in many applications, such as designing flight paths and missile batteries [2], allocating communication facilities [3], locating wind turbines [4], analyzing archaeological locations [5].

In this paper, we concentrate on how to accelerate Viewshed Analysis processing under the General Purpose Graphical Processing Units (GPGPU) architecture. We will describe algorithms for Viewshed Analysis and Compute Unified Device Architecture (CUDA) in brief. And we will discuss in detail about the strategies we employed to assign the data to different memory spaces on GPU chip, to regularize the computing instructions and to minimize branch parts in the procedure.

Several algorithms have been raised for viewshed analysis, like the point-to-point method [6], vector-analysis method [7], local surface method [8], and reference plane method [9]. In these algorithms, different measures were made to reach the compromise between the performance and accuracy [10]. The most accurate solution is still the basic interpolation method, which makes full use of the available elevation data and is considered as the standard method. However, the time cost is N^3 for an N by N grid, which is too huge especially for big scale data.

The recent development of GPU enables normal PC's computing power to make significant progress. Different from traditional CPU with most of its transistors used for caching, modern GPU reduce the proportion of caching transistors and increase the proportion for computing operational circuit. So, a normal GPU has hundreds of independent processor cores and has relatively low cost per unit of floating-point (FP) performance. Therefore, computational scientists have long been interested in GPU for general purpose computing. In 2007, NVIDIA Corporation released Compute Unified Device Architecture (CUDA) which could help us to deploy some computing sensitive applications into GPU environment and make full use of GPU' s parallel computing ability. Some experiments were conducted in computing sensitive areas and geographical information processing [11-14].

This paper is organized as follows: we first describe in more detail about the basic interpolation method and why we employ it. Then we introduce the computing model of CUDA and list the problems we met when mapping the application to the GPU architecture. We then give our solutions to solve the problems and illustrate the performance of our solutions. We will close with some future work.

II. METHODS FOR VIEWSHED ANALYSIS

The basic algorithm is to calculate the visibility of each cell from view point separately. And the visibility is determined by whether the line joining view point and target cell, namely the sight line, intersects the digital elevation model. Suppose that we have a view point named P_v, a target point named P_t, and a sightline ray named L_s joining P_v and P_t. For an arbitrary point P along L_s. Whether the sightline ray can be obstructed by point P can be calculated by the following formula:

$$obs(p) = elev(p_v) + \frac{dis(p, p_v)}{dis(p_t, p_v)}(elev(p_t) - elev(p_v)) - elev(p)$$

Where obs(p) means whether the sightline is obstructed by point P, elev(p_v) means the elevation of point P_v, and dis(p, p_v) means the distance between point P and P_v. If obs(p) is negative, the sightline ray rips into the terrain elevation surface and the target point P_t is invisible from view point. Otherwise, target point is not obstructed by point P and we have to check other points in the sightline using this formula. Strictly

Supported by the National Hi-Tech Research and Development 863 Program of China (Grant No. 2011AA120301)

978-1-61284-848-8/11/$26.00 ©2011 IEEE

speaking, only when all the points between view point and target point do not obstruct the sightline, can the target point be definitely visible from view point.

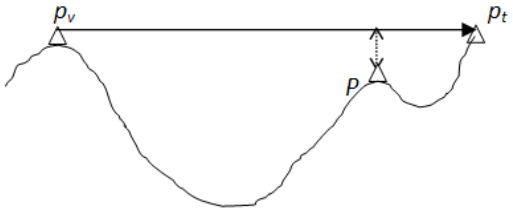


Figure 1. Viewshed Analysis

However, the points in the continuum between view point and target point along the sightline are infinite and in practice, we need a sampling schema to choose points along the sightline and check whether the points obstruct the target point. A usual sampling schema is to choose the points where the sightline intersects with X-gridlines or Y-gridlines, as in Figure 2. There are other simplifications of this sampling strategy, like choosing points just along X or Y coordinates. However, they are less accurate than the basic method, and in this paper, we still take the densely basic sampling strategy to get more accurate results. The elevation of chosen point is calculated by bilinear interpolation of elevation of nearby grid cross points. Based on the method above, we can compute every cell's visibility.

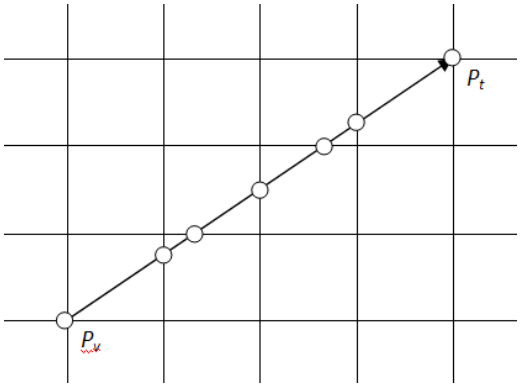


Figure 2. Choose the points where the sightline intersects with X-gridlines or Y-gridlines

There also exist many other methods for Viewshed Analysis, as we mentioned in section one. By reusing the intermediate results to calculate each cell's visibility proximately, some of them have much less time cost. However, in some cases, the results are not that accurate. Taking the reference plane method as an example, there is a sub-matrix data from a DEM as showed in Figure 3. where elev(P_v)=100, elev(A)=100, elev(B)=103, elev(C)=103, elev(D)=103, elev(E)=105. If we employ the basic interpolation method, the interpolations for point F and G are 101 and 103, while the critical elevation for F and G are 101.6 and 103.3. So the point E is not obstructed by point F or G and is visible under condition of the basic interpolation. However, if we employ the reference plane method, the result is different. First, point A and B are visible. Second, the critical elevation for C is 100 by reference plane P_v-A-B, which is lower than 103, point C's actual elevation. Point C is visible. Third, the critical elevation for D is 104.5 by reference plane P_v-B-C, which is higher than 103, point D's actual elevation. Point D is invisible. At last, the critical elevation for E is 106, which is higher than 105, point E's actual elevation. Point E is invisible. Now we can see that the results are already different between these two methods. The visibilities of points after point E along sightline will be strongly influenced by the point E's visibility and the difference would be huge.

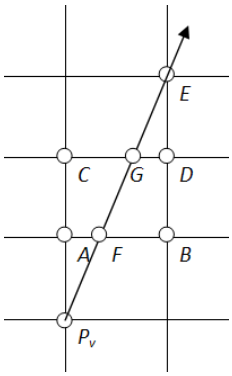


Figure 3. Example of difference between reference plane method and basic interpolation method

Reusing the intermediate values to get a proximately results, like reference plane method, is actually a compromise between the performance and accuracy [15]. Therefore, Different methods are usually employed according to different demands on performance and accuracy. Generally speaking, we hold that, except for some real time areas, accuracy deserves more attention, and the effects of time cost will be reduced by the development of computing technology. In this paper, we take the basic interpolation as the default method to get more accurate results.

III. INTRODUCTION OF CUDA

In this section, we will introduce the memory model and threading and instruction model of CUDA.

Variables in CUDA application should be stored in device memory spaces on GPU chip. There are different memory spaces with different accessing characteristics for different distinct usages in CUDA applications. These memories are divided into global, local, constant, texture, register and shared memory (Figure 4). In the capacity perspective, the global and texture memory are most plentiful. While local memory is limited to 16 KB per thread, constant memory is limited to 64 KB in total, shared memory is limited to 16 KB per multiprocessor, and 32-bit register memory is limited to 8,192 or 16384 per multiprocessor [16]. In access efficiency perspective, the registers, constant and shared memory have the least access latency with 24 cycles. The global, local and texture memory's access latency is about 400 to 600 cycles. Memory optimizations are the most important area for CUDA performance. The key point is to use as much fast memory and as little slow-access memory as possible.

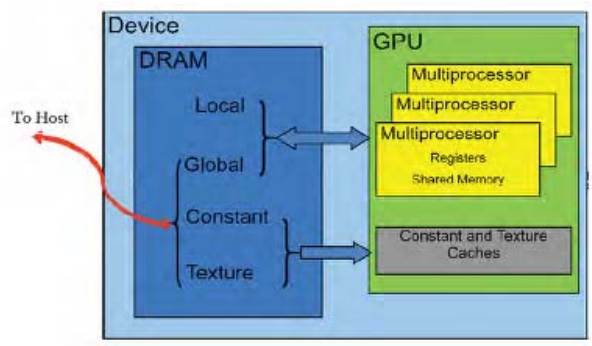


Figure 4. CUDA memory model

A certain code segment executing on different data sets can be isolated to a function, which is called a kernel. In a kernel, threads are organized to grids, blocks and threads. A grid has many blocks, and the blocks cannot communicate with each other. A block has less than 512 threads, and the threads can communicate and synchronize with each other. Each block of threads are assigned to one Stream multiprocessor (SM) and split into two or three warps. Each warp has eight threads. Warps are executed by SM in Simple Instruction Multiple Threads (SIMT) fashion. The thread scheduler switches from warp to warp to maximize the processor efficiency.

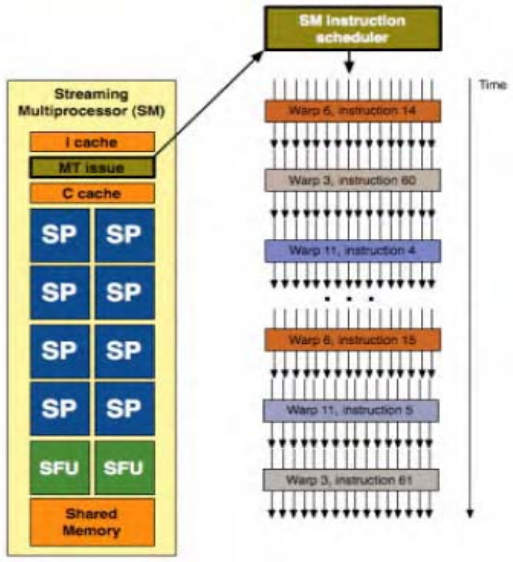


Figure 5. CUDA thread model

IV. Parallel Analysis and Implementations

In this section, we will analyze feasibility of mapping the basic interpolation method into CUDA environment and illustrate problems we met during this procedure. Then we give our strategies for data assigning and instruction optimization on GPU chip using CUDA.

In parallel perspective, no dependency is found between each cell's visibilities in the basic interpolation method, which make it possible for parallel processing. On the other hand, the data in grid DEM are arranged in regular matrix form, which is suitable for GPU computing. However, the disadvantage is that the computation in Viewshed Analysis mainly focuses on the interpolation between sight line and elevation matrix, which is irregular along sight line in traditional CPU procedure for different sight lines have different execution branches and cost different time.

A new strategy is needed to minimize this irregularity in GPU environment. Specially, there are two problems mapping the Viewshed Analysis algorithm into CUDA application to get better performance. The first one is how to assign data into different memory spaces on GPU chip when taking consideration of different memory access efficiency. The second is how to regularize the computing instructions and minimize the branch parts between threads from a block, for they share the same instruction front end and only regularized and consistent instructions can take full advantage of hundreds of executing cores on GPU.

To the first problem, we assign the data into different memory spaces, according to their different access latencies and capacities. The origin grid DEM data are put in texture memory as CUDA Arrays. On one hand, texture memory has plentiful capacity for the DEM data. On the other hand, it has texture fetch pipeline providing caches to support irregular data accesses. Information about view point, such as location coordinates and height, is put in constant memory. Data in constant memory are unwritable after initialization and can be read simultaneously in one memory transaction by threads of a half-warp. So different executing cores can simultaneously read the view point's location coordinates and save a lot of both time and memory. The result matrix data are assigned into global memory. The global memory has plentiful capacity for the data matrix but the data are not cached. However, global memory access by all threads of a half-warp is coalesced into one or two memory transactions following the right access pattern to get maximum memory bandwidth. Finally, the temporary variables in the procedure are stored in registers.

To the second problem, we proposed a new traversal method named the "Comb" method. Traditional CPU method computes each cell's visibility serially. CPU processes only one cell's visibility at a time and computes the interpolation along the sight line. While on GPU, with the "Comb" method, we divide the basic interpolation along the sight line into two parts: interpolation along the X coordinate direction and along the Y coordinate direction. Then, we combine all the X direction interpolation from threads of one block together into one part, Y direction to the other part. "Comb" along X coordinate direction means all the threads in a block do interpolation job with Y coordinate lines in the X direction. Two optimization measures are employed here. First, we set an increment variable, 1, 0 or -1, at the beginning according to the target cell's relative position to view point, before, equal or later. With this variable, the later instruction fragment from threads in a block will be the same. Second, we set synchronization before the loop, so the processors within a SM will execute the same instructions.

The work flow of our viewshed analysis method using CUDA is divided into following 10 steps as Figure 6. The red

ones are executed in kernel function. In addition, the strategy we take only changes the computing order, but the result will be the same with origin basic interpolation.

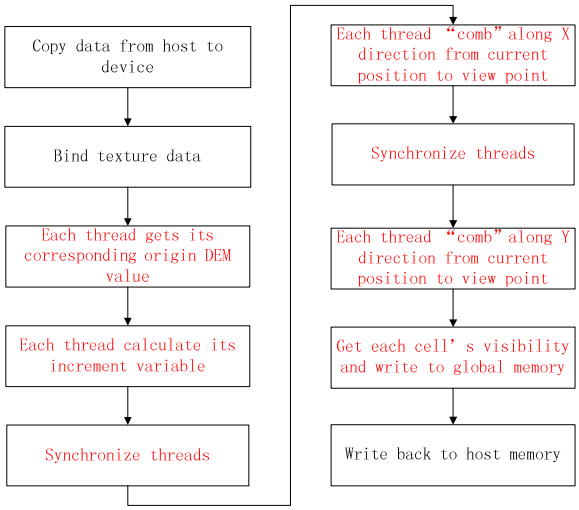


Figure 6. steps in the viewshed analysis using CUDA

V. EXPERIMENTS AND RESULTS

We carried out an experiment to study the new method's performance on GPU compared with CPU. The experiment was carried out on one computer. The CPU we used was "*Intel (R) Core (TM)2 Quad CPU Q9550 @2.83GHz*", while the GPU was "*NVIDIA GeForce GT 240*" with CUDA version 2.3. On the GPU chip we employed, there were 128 stream processors totally. Two DEM data were employed here. They are DEM A and DEM B. DEM A had 2847*3717 cells. DEM B had 366 * 372 cells. For DEM A, the position of view point was (150, 150) and the height is 1000 inches. For DEM B, the position of view point was (150, 150) with height of 800 inches. The difference of time cost between CPU and GPU are shown as table 1.

TABLE I. RESULTS OF THE EXPERIMENT

Time cost compare	***DEM A***	***DEM B***
Time cost on CPU	46390 ms	797 ms
Time cost on GPU	656 ms	47 ms

We can see that, for small scale DEM data, the speedup is lower, such as DEM B with about 17 times' speedup. While for large scale data, the speedup is higher, such as DEM A with about 70 times' speedup. This is because extra time is cost for data transferring between host and device memory in GPU applications. And it occupies a big part for small scale data and relatively smaller part for larger scale data.

VI. CONCLUSION

A new strategy for mapping viewshed analysis application into GPU environment is proposed in this paper. Under the premise to ensure the result's accuracy, we choose the basic interpolation method to accelerate, for it costs too much time on CPU. We give two solutions to optimize DEM processing on GPU. The first is for data assignment into different memory spaces on GPU chip: global variables are stored in constant memory, DEM matrix data are stored in texture memory and results are written into global memory. The second is instruction optimization for SM on GPU. By the "comb" method, we divide the computing job for interpolation into two parts: computing along X direction and Y direction. Thus, the computing instructions are regularized. By setting the increment variables, we minimize branch parts of different threads.

The experiment shows that, the strategy we employ get notable speedups, especially for large scale grid DEM data. However, there is an issue to be settled that: when the data scale is too large, the device memory on GPU is not enough for the whole data. So, a new data partition strategy for viewshed analysis on GPU is needed in future work.

REFERENCES

[1] Viewshed Analysis. In *Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Viewshed_Analysis&oldid=394923131, November 5, 2010.

[2] W.R. Franklin, C.K. Ray, and S. Mehta, "Geometric algorithms for sitting of air defense missile batteries," Research Project for Battelle, Columbus Division, Contract Number DAAL03-86-D-0001, Delivery Order No. 2756, 1994.

[3] J. Lee, "Analyses of visibility sites on topographic surfaces," International Journal of Geographical Information Systems, 4, pp. 413–429, 1991.

[4] D. Kidner, A. Sparkes, M. Dorey, "GIS and wind farm planning," In: Stillwell, J., Geertman, S., Openshaw, S. (Eds.), Geographical Information and Planning. Springer, London, pp. 203–223, 1999.

[5] M.W. Lake, P.E. Woodman, S.J. Mithen, "Tailoring GIS software for archaeological applications: an example concerning viewshed analysis.," Journal of Archaeological Science 25, 27–38, 1998.

[6] R.E. Huss and M.A. Pumar, "Effect of database errors on intervisibility estimation," Photogrammetric Engineering and Remote Sensing, 63, pp. 415–424. 1997.

[7] P.A. Sorensen and D.P. Lanter, "Two algorithms for determining partial visibility and reducing data structure induced error in viewshed analysis," Photogrammetric Engineering and Remote Sensing, 59, pp. 1149–1160, 1993.

[8] J. Wang, G.J. Robinson and K. White, "A fast solution to local viewshed computation using grid-based digital elevation models," Photogrammetric Engineering and Remote Sensing, 62, pp. 1157–1164, 1996.

[9] J. Wang, G.J. Robinson and K. White, "Generating viewsheds without using sightlines," Photogrammetric Engineering and Remote Sensing, 66, pp. 87–90, 2000.

[10] D. Izraelevitz, "A fast algorithm for approximate viewshed computation.," Photogrammetric Engineering and Remote Sensing, 69, pp. 767–774, 2003.

[11] Y. Tan, W. Tan, and L. Kwoh, "Fast Colour Balance Adjustment of IKONOS Imagery Using CUDA," 2008 IEEE International Geoscience & Remote Sensing Symposium, pp. II-1052-II-1055, 2008.

[12] Y. Zhao, Z. Huang, B. Chen, Y. Fang, M.L. Yan, Z. Yang, "Local Acceleration in Distributed Geographic Information Processing with CUDA," *Geoinformatics, 2010 18th International Conference on* , vol., no., pp.1-6, 18-20 June 2010.

[13] R. Membarth, F. Hannig, H. Dutta, and J. Teich, "Efficient Mapping of Multiresolution Image Filtering Algorithms on Graphics Processors," Proceedings of the 9th International Workshop on Embedded Computer Systems: Architectures, Modeling, and Simulation, pp. 277-288, 2009.

[14] X. Sun, M. Li, Y. Liu, L. Tan, and W. Liu, “Accelerated segmentation approach with CUDA for high spatial resolution remotely sensed imagery based on improved Mean Shift,” Urban Remote Sensing Event, 2009 Joint, pp.1-6, 2009.

[15] W. Huanping, P. Mao, Y. Lingqing and L. Bing, “A partition-based serial algorithm for generating viewshed on massive DEMs,” International Journal of Geographical Information Science, 21(9): 955-964 , 2007.

[16] NVIDIA, “NVIDIA CUDA Best Practices Guide, Version 3.0”, http://developer.nvidia.com/object/cuda_3_0_downloads.html, 2010.